

Techathon-IUT Hackathon — Complete Solution Guide

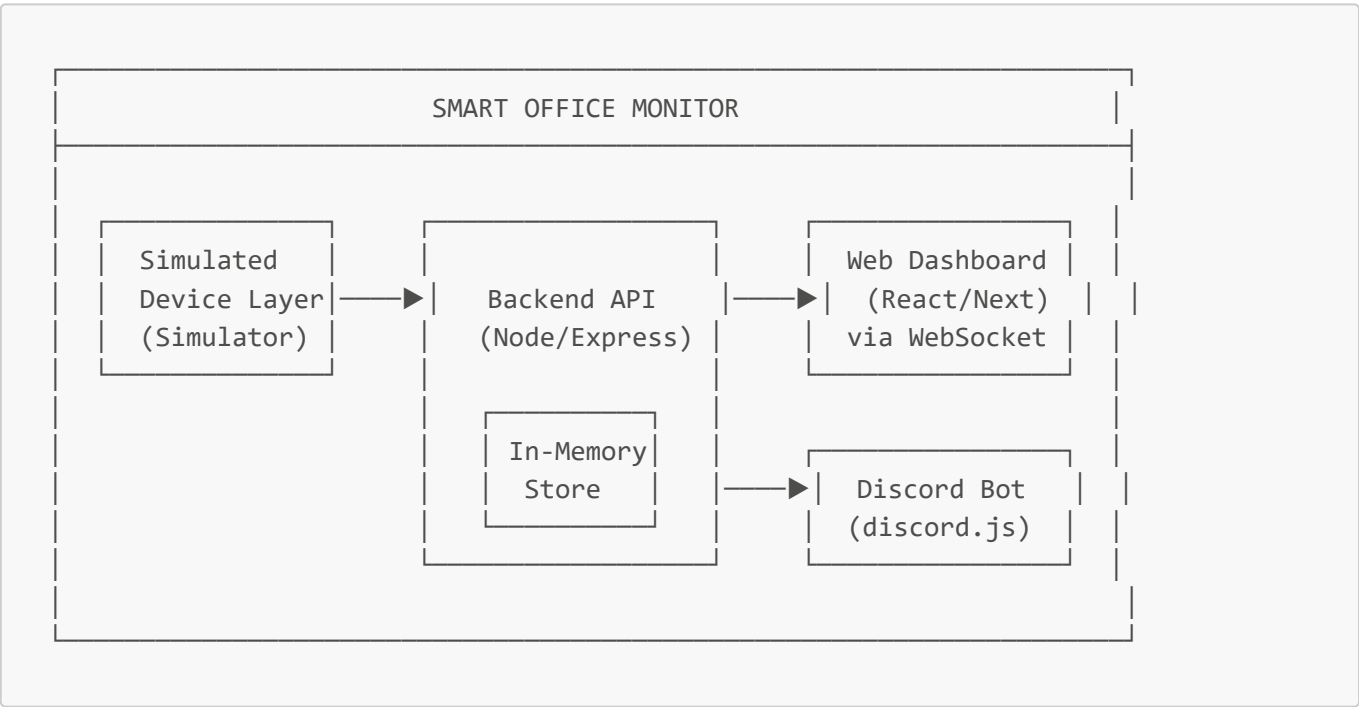
Problem Summary

Build a **Smart Office Monitoring System** that tracks lights & fans across 3 rooms (18 devices total) via:

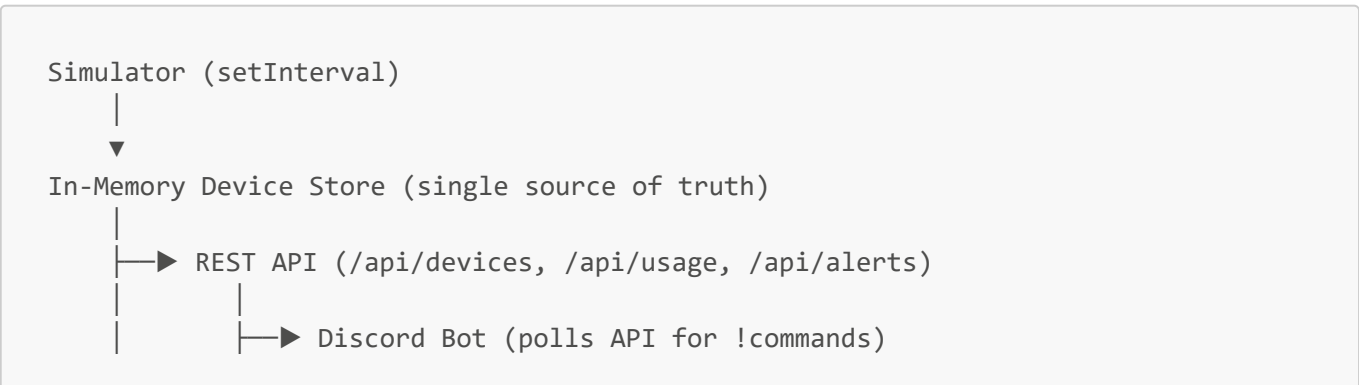
- 1. **System Diagram** — full data-flow architecture (15%)
- 2. **Hardware Schematic** — Wokwi/Tinkercad circuit for 1 room (15%)
- 3. **Simulated Device Data** — dynamic dummy data for all 18 devices (15%)
- 4. **Web Dashboard** — real-time UI with live status, power meter, alerts (20% + 10% UX)
- 5. **Discord Bot** — `!status`, `!room <name>`, `!usage` commands pulling live data (10%)
- 6. **Clean Codebase & Docs** (15%)

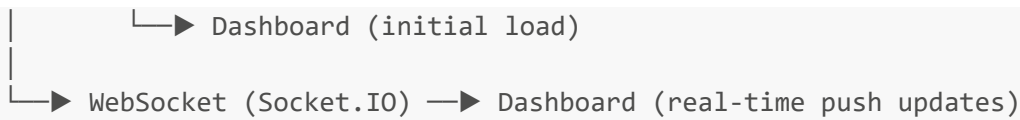
Key Constraint: Dashboard + Discord Bot share a **single backend** (one source of truth).

High-Level Architecture



Data Flow





📁 Recommended Project Structure

```

Techathon-IUT/
├── backend/
│   ├── server.js           # Express + Socket.IO server
│   ├── simulator.js        # Device data simulator
│   ├── deviceStore.js      # In-memory device state manager
│   ├── routes/
│   │   ├── devices.js      # GET /api/devices, /api/devices/:room
│   │   ├── usage.js        # GET /api/usage
│   │   └── alerts.js        # GET /api/alerts
│   ├── utils/
│   │   ├── alertEngine.js  # Alert detection logic
│   │   └── powerCalc.js    # Power calculation helpers
│   ├── package.json
│   └── .env
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── components/
│   │   │   ├── DevicePanel.jsx    # Live device status grid
│   │   │   ├── PowerMeter.jsx    # Total + per-room power gauge
│   │   │   ├── AlertsPanel.jsx    # Active alerts list
│   │   │   ├── OfficeFloorPlan.jsx # Top-view layout (BONUS)
│   │   │   ├── RoomCard.jsx       # Per-room device summary
│   │   │   └── DeviceIcon.jsx     # Fan animation / light glow
│   │   ├── hooks/
│   │   │   └── useSocket.js       # WebSocket hook
│   │   ├── styles/
│   │   │   └── index.css
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
├── discord-bot/
│   ├── bot.js              # Discord bot entry point
│   ├── commands/
│   │   ├── status.js       # !status command
│   │   ├── room.js         # !room <name> command
│   │   └── usage.js         # !usage command
│   ├── utils/
│   │   └── apiClient.js     # Axios wrapper to call backend API
│   ├── package.json
│   └── .env
├── diagrams/
│   └── system-diagram.png  # High-level architecture diagram
  
```

```
| └─ circuit-schematic.png # Wokwi/Tinkercad screenshot
| └─ hackathon.md          # This file
| └─ README.md             # Setup & run instructions
```

Step-by-Step Implementation

Step 1: Backend — Device Store & Simulator (Core Foundation)

Priority: HIGH — Everything depends on this. Build it first.

1.1 Initialize Backend

```
mkdir backend && cd backend
npm init -y
npm install express socket.io cors dotenv
```

1.2 Device Store (`deviceStore.js`)

This is the **single source of truth** for all device states.

```
// backend/deviceStore.js

const ROOMS = ['Drawing Room', 'Work Room 1', 'Work Room 2'];
const DEVICE_TYPES = [
  { type: 'fan', count: 2, powerWatt: 60 },
  { type: 'light', count: 3, powerWatt: 15 },
];

class DeviceStore {
  constructor() {
    this.devices = [];
    this._initDevices();
  }

  _initDevices() {
    let id = 1;
    for (const room of ROOMS) {
      for (const { type, count, powerWatt } of DEVICE_TYPES) {
        for (let i = 1; i <= count; i++) {
          this.devices.push({
            id: id++,
            name: `${type.charAt(0).toUpperCase() + type.slice(1)} ${i}`,
            type,
            room,
            status: Math.random() > 0.5 ? 'on' : 'off',
            powerWatt,
          });
        }
      }
    }
  }
}
```

```
        currentDraw: 0, // calculated based on status
        lastChanged: new Date().toISOString(),
    });
    }
}
}
this._recalcPower();
}

_recalcPower() {
    for (const device of this.devices) {
        device.currentDraw = device.status === 'on' ? device.powerWatt : 0;
    }
}

toggleDevice(deviceId) {
    const device = this.devices.find(d => d.id === deviceId);
    if (!device) return null;
    device.status = device.status === 'on' ? 'off' : 'on';
    device.currentDraw = device.status === 'on' ? device.powerWatt : 0;
    device.lastChanged = new Date().toISOString();
    return device;
}

getAllDevices() {
    return this.devices;
}

getDevicesByRoom(roomName) {
    return this.devices.filter(
        d => d.room.toLowerCase().replace(/\s/g, '') ===
roomName.toLowerCase().replace(/\s/g, '')
    );
}

getTotalPower() {
    return this.devices.reduce((sum, d) => sum + d.currentDraw, 0);
}

getPowerByRoom() {
    const result = {};
    for (const room of ROOMS) {
        result[room] = this.devices
            .filter(d => d.room === room)
            .reduce((sum, d) => sum + d.currentDraw, 0);
    }
    return result;
}

getEstimatedDailyUsage() {
    // Estimate based on current draw extrapolated over 8 office hours
    const totalWatts = this.getTotalPower();
    const kWh = (totalWatts * 8) / 1000;
    return Math.round(kWh * 10) / 10;
}
```

```
}  
}  
  
module.exports = new DeviceStore();
```

1.3 Simulator (simulator.js)

Randomly toggles devices every few seconds to create "live" feel.

```
// backend/simulator.js  
  
const deviceStore = require('./deviceStore');  
  
function startSimulator(io) {  
  // Toggle a random device every 5-10 seconds  
  setInterval(() => {  
    const devices = deviceStore.getAllDevices();  
    const randomDevice = devices[Math.floor(Math.random() * devices.length)];  
    const updated = deviceStore.toggleDevice(randomDevice.id);  
  
    // Push update to all connected web dashboard clients  
    if (io && updated) {  
      io.emit('deviceUpdate', {  
        device: updated,  
        allDevices: deviceStore.getAllDevices(),  
        totalPower: deviceStore.getTotalPower(),  
        powerByRoom: deviceStore.getPowerByRoom(),  
        alerts: require('./utils/alertEngine').getAlerts(deviceStore),  
      });  
    }  
  
    console.log(`[Simulator] Toggled ${updated.name} in ${updated.room} →  
    ${updated.status}`);  
    }, Math.floor(Math.random() * 5000) + 5000);  
  }  
  
  module.exports = { startSimulator };
```

1.4 Alert Engine (utils/alertEngine.js)

```
// backend/utils/alertEngine.js  
  
function getAlerts(deviceStore) {  
  const alerts = [];  
  const now = new Date();  
  const currentHour = now.getHours();  
  const isAfterHours = currentHour < 9 || currentHour >= 17;
```

```

const devices = deviceStore.getAllDevices();

// Alert 1: Devices ON after office hours (before 9 AM or after 5 PM)
if (isAfterHours) {
  const onDevices = devices.filter(d => d.status === 'on');
  if (onDevices.length > 0) {
    const roomGroups = {};
    for (const d of onDevices) {
      if (!roomGroups[d.room]) roomGroups[d.room] = [];
      roomGroups[d.room].push(d.name);
    }
    for (const [room, devNames] of Object.entries(roomGroups)) {
      alerts.push({
        id: `afterhours-${room}`,
        type: 'after-hours',
        severity: 'warning',
        message: `⚠️ ${room} has ${devNames.length} device(s) ON after office
hours: ${devNames.join(', ')}`,
        timestamp: now.toISOString(),
        room,
      });
    }
  }
}

// Alert 2: All devices in a room ON for 2+ hours
const rooms = ['Drawing Room', 'Work Room 1', 'Work Room 2'];
for (const room of rooms) {
  const roomDevices = devices.filter(d => d.room === room);
  const allOn = roomDevices.every(d => d.status === 'on');
  if (allOn) {
    const oldest = roomDevices.reduce((min, d) =>
      new Date(d.lastChanged) < new Date(min.lastChanged) ? d : min
    );
    const hoursOn = (now - new Date(oldest.lastChanged)) / (1000 * 60 * 60);
    if (hoursOn >= 2) {
      alerts.push({
        id: `allOn-${room}`,
        type: 'all-devices-on',
        severity: 'critical',
        message: `🔴 All devices in ${room} have been ON for
${Math.round(hoursOn * 10) / 10} hours!`,
        timestamp: now.toISOString(),
        room,
      });
    }
  }
}

return alerts;
}

module.exports = { getAlerts };

```

1.5 Express Server (`server.js`)

```
// backend/server.js

require('dotenv').config();
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
const deviceStore = require('./deviceStore');
const { getAlerts } = require('./utils/alertEngine');
const { startSimulator } = require('./simulator');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: '*' }
});

app.use(cors());
app.use(express.json());

// — REST API Routes —

// Get all devices
app.get('/api/devices', (req, res) => {
  res.json({
    devices: deviceStore.getAllDevices(),
    totalPower: deviceStore.getTotalPower(),
    powerByRoom: deviceStore.getPowerByRoom(),
  });
});

// Get devices by room
app.get('/api/devices/:room', (req, res) => {
  const roomName = req.params.room;
  const devices = deviceStore.getDevicesByRoom(roomName);
  if (devices.length === 0) {
    return res.status(404).json({ error: `Room '${roomName}' not found` });
  }
  res.json({ room: roomName, devices });
});

// Get power usage
app.get('/api/usage', (req, res) => {
  res.json({
    totalPowerWatts: deviceStore.getTotalPower(),
    powerByRoom: deviceStore.getPowerByRoom(),
    estimatedDailyKWh: deviceStore.getEstimatedDailyUsage(),
  });
});
```

```
// Get active alerts
app.get('/api/alerts', (req, res) => {
  res.json({ alerts: getAlerts(deviceStore) });
});

// — WebSocket —————

io.on('connection', (socket) => {
  console.log(`[WS] Client connected: ${socket.id}`);
  // Send initial state on connect
  socket.emit('initialState', {
    devices: deviceStore.getAllDevices(),
    totalPower: deviceStore.getTotalPower(),
    powerByRoom: deviceStore.getPowerByRoom(),
    alerts: getAlerts(deviceStore),
  });

  socket.on('disconnect', () => {
    console.log(`[WS] Client disconnected: ${socket.id}`);
  });
});

// — Start —————

const PORT = process.env.PORT || 4000;
server.listen(PORT, () => {
  console.log(`🚀 Backend running on http://localhost:${PORT}`);
  startSimulator(io);
});
```

Step 2: Web Dashboard (React + Vite)

Priority: HIGH — Worth 30% (20% functionality + 10% UX/visuals)

2.1 Initialize Frontend

```
npx -y create-vite@latest frontend -- --template react
cd frontend
npm install socket.io-client axios
npm install lucide-react # for icons (fans, lights, alerts)
```

2.2 Key Components to Build

useSocket.js — **WebSocket Hook**


```
// frontend/src/hooks/useSocket.js
import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';

const BACKEND_URL = 'http://localhost:4000';

export function useSocket() {
  const [data, setData] = useState({
    devices: [],
    totalPower: 0,
    powerByRoom: {},
    alerts: [],
  });

  useEffect(() => {
    const socket = io(BACKEND_URL);

    socket.on('initialState', (state) => setData(state));
    socket.on('deviceUpdate', (update) => {
      setData({
        devices: update.allDevices,
        totalPower: update.totalPower,
        powerByRoom: update.powerByRoom,
        alerts: update.alerts,
      });
    });

    return () => socket.disconnect();
  }, []);

  return data;
}
```

Dashboard Layout

The dashboard should have these sections:

Section	Description	Visual
Header	Office name, current time, total power badge	Glassmorphism bar
Room Cards	3 cards showing each room's devices	Grid of device icons with ON/OFF glow
Power Meter	Total wattage + per-room breakdown	Animated gauge / progress bars
Alerts Panel	Active alerts with timestamps	Red/amber cards with pulse animation

Section	Description	Visual
Floor Plan (BONUS)	Top-view SVG of office layout	Lights glow, fans spin when ON

Winning UX Tips 🏆

- 1. **Fan Animation:** Use CSS `@keyframes spin` on fan icons when `status === 'on'`
- 2. **Light Glow:** Use `box-shadow` with a yellow/warm glow when light is ON
- 3. **Live Pulse:** Add a subtle pulsing dot next to "LIVE" indicator
- 4. **Color Coding:** Green = all off, Amber = some on, Red = alert state
- 5. **Dark Theme:** Dark background (#0f172a) with vibrant accent colors
- 6. **Smooth Transitions:** `transition: all 0.3s ease` on state changes

CSS Animation Examples

```
/* Fan spinning animation */
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
.fan-on {
  animation: spin 1s linear infinite;
}

/* Light glow effect */
.light-on {
  box-shadow: 0 0 20px 8px rgba(255, 200, 50, 0.6);
  background: radial-gradient(circle, #fff7cc, #ffd700);
}

/* Alert pulse */
@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}
.alert-badge {
  animation: pulse 2s ease-in-out infinite;
}

/* Live indicator dot */
.live-dot {
  width: 8px;
  height: 8px;
  background: #22c55e;
  border-radius: 50%;
  animation: pulse 1.5s ease-in-out infinite;
}
```

2.3 Floor Plan (BONUS — High Impact)

Create an SVG-based top-view office layout:

```
// Simplified concept – use SVG elements for each room
<svg viewBox="0 0 800 400" className="office-floor-plan">
  {/* Drawing Room */}
  <rect x="10" y="10" width="250" height="380" rx="8" fill="#1e293b"
stroke="#334155" />
  <text x="135" y="40" textAnchor="middle" fill="#e2e8f0">Drawing Room</text>

  {/* Lights – circle elements that change fill based on status */}
  {drawingRoomLights.map((light, i) => (
    <circle
      key={light.id}
      cx={60 + i * 80}
      cy={100}
      r={12}
      fill={light.status === 'on' ? '#fbbf24' : '#475569'}
      filter={light.status === 'on' ? 'url(#glow)' : 'none'}
    />
  ))}

  {/* Fans – animated SVG groups */}
  {drawingRoomFans.map((fan, i) => (
    <g key={fan.id} className={fan.status === 'on' ? 'fan-on' : ''}>
      {/* fan blade SVG paths */}
    </g>
  ))}

  {/* Work Room 1 & 2 similarly */}
</svg>
```

Step 3: Discord Bot

Priority: MEDIUM — Worth 10%, but easy to impress with conversational responses

3.1 Setup

```
mkdir discord-bot && cd discord-bot
npm init -y
npm install discord.js axios dotenv
```

3.2 Create Discord Application

1. Go to [Discord Developer Portal](#)
2. Click **New Application** → Name it "Office Monitor Bot"

3. Go to **Bot** → Click **Add Bot** → Copy the **TOKEN**
4. Enable **Message Content Intent** under Privileged Gateway Intents
5. Go to **OAuth2** → **URL Generator** → Select **bot** scope → Select **Send Messages, Read Message History** permissions
6. Use the generated URL to invite bot to your server

3.3 Bot Implementation (bot.js)

```
// discord-bot/bot.js

require('dotenv').config();
const { Client, GatewayIntentBits } = require('discord.js');
const axios = require('axios');

const client = new Client({
  intents: [
    GatewayIntentBits.Guilds,
    GatewayIntentBits.GuildMessages,
    GatewayIntentBits.MessageContent,
  ],
});

const API_BASE = process.env.API_BASE || 'http://localhost:4000/api';

// — Command Handlers —————

async function handleStatus(message) {
  try {
    const { data } = await axios.get(`${API_BASE}/devices`);
    const rooms = {};
    for (const device of data.devices) {
      if (!rooms[device.room]) rooms[device.room] = { fans: [], lights: [] };
      rooms[device.room][device.type === 'fan' ? 'fans' : 'lights'].push(device);
    }

    let reply = '🏢 **Office Status Report**\n\n';
    for (const [room, devices] of Object.entries(rooms)) {
      const fansOn = devices.fans.filter(d => d.status === 'on').length;
      const lightsOn = devices.lights.filter(d => d.status === 'on').length;
      const emoji = (fansOn + lightsOn === 0) ? '🕸' : '🕹';
      reply += `${emoji} **${room}**: ${fansOn} fan(s) ON, ${lightsOn} light(s) ON\n`;
    }
    reply += `\n⚡ Total Power: **${data.totalPower}W**`;
    message.reply(reply);
  } catch (err) {
    message.reply('❌ Oops! Couldn\'t reach the backend. Is it running?');
  }
}

async function handleRoom(message, roomArg) {
```

```

const roomMap = {
  'drawing': 'DrawingRoom',
  'drawingroom': 'DrawingRoom',
  'work1': 'WorkRoom1',
  'workroom1': 'WorkRoom1',
  'work2': 'WorkRoom2',
  'workroom2': 'WorkRoom2',
};

const roomKey = roomMap[roomArg.toLowerCase().replace(/\s/g, '')];
if (!roomKey) {
  return message.reply('🤖 I don\'t know that room! Try: `drawing`, `work1`, or `work2`');
}

try {
  const { data } = await axios.get(`${API_BASE}/devices/${roomKey}`);
  let reply = `🏠 **${data.room} Details**\n\n`;
  for (const device of data.devices) {
    const icon = device.type === 'fan' ? '🌀' : '💡';
    const status = device.status === 'on' ? '✅ ON' : '❌ OFF';
    const power = device.status === 'on' ? `(${device.powerWatt}W)` : '';
    reply += `${icon} ${device.name}: ${status} ${power}\n`;
  }
  message.reply(reply);
} catch (err) {
  message.reply('❌ Couldn\'t find that room. Try: `drawing`, `work1`, or `work2`');
}

async function handleUsage(message) {
  try {
    const { data } = await axios.get(`${API_BASE}/usage`);
    let reply = '⚡ **Power Usage Report**\n\n';
    reply += `📊 Total power right now: **${data.totalPowerWatts}W**\n`;
    reply += `📈 Today's estimated usage: **${data.estimatedDailyKWh} kWh**\n\n`;
    reply += `🏠 **Per-Room Breakdown:**\n`;
    for (const [room, watts] of Object.entries(data.powerByRoom)) {
      reply += `    • ${room}: ${watts}W\n`;
    }
    message.reply(reply);
  } catch (err) {
    message.reply('❌ Couldn\'t fetch usage data. Is the backend running?');
  }
}

// — Message Listener —————

client.on('messageCreate', async (message) => {
  if (message.author.bot) return;

  const content = message.content.trim();

```

```
    if (content === '!status') return handleStatus(message);
    if (content.startsWith('!room ')) return handleRoom(message,
content.slice(6).trim());
    if (content === '!usage') return handleUsage(message);
  });

// — BONUS: Proactive Alert Posting —————

async function checkAndPostAlerts() {
  try {
    const { data } = await axios.get(`${API_BASE}/alerts`);
    if (data.alerts.length > 0) {
      const channel = client.channels.cache.get(process.env.ALERT_CHANNEL_ID);
      if (channel) {
        for (const alert of data.alerts) {
          channel.send(alert.message);
        }
      }
    }
  } catch (err) {
    console.error('Alert check failed:', err.message);
  }
}

// Check for alerts every 5 minutes
setInterval(checkAndPostAlerts, 5 * 60 * 1000);

// — Login —————

client.login(process.env.DISCORD_TOKEN);
```

3.4 Environment Variables

```
# discord-bot/.env
DISCORD_TOKEN=your_bot_token_here
API_BASE=http://localhost:4000/api
ALERT_CHANNEL_ID=your_channel_id_here
```

Step 4: Hardware Schematic (Wokwi)

Priority: MEDIUM — Worth 15%, concept-only (no real hardware)

Recommended: Wokwi (works better with AI-assisted dev)

Circuit Design for ONE Room (2 fans, 3 lights)

Component	Wokwi Part	Pin	Purpose
-----------	------------	-----	---------

Component	Wokwi Part	Pin	Purpose
ESP32	board-esp32-devkit-c-v4	—	Microcontroller
LED 1 (Light 1)	wokwi-led	GPIO 13	Simulates Light 1
LED 2 (Light 2)	wokwi-led	GPIO 12	Simulates Light 2
LED 3 (Light 3)	wokwi-led	GPIO 14	Simulates Light 3
DC Motor 1 (Fan 1)	wokwi-led (or motor)	GPIO 27	Simulates Fan 1
DC Motor 2 (Fan 2)	wokwi-led (or motor)	GPIO 26	Simulates Fan 2
ACS712 Current Sensor	Analog input	GPIO 34 (ADC)	Measures total current
Relay Module (x5)	wokwi-led	GPIO 25,33,32,35,23	Controls each device
220Ω Resistors (x5)	wokwi-resistor	In series with LEDs	Current limiting

Wiring Logic

```
ESP32 GPIO 13 → 220Ω Resistor → LED 1 (Light 1) → GND
ESP32 GPIO 12 → 220Ω Resistor → LED 2 (Light 2) → GND
ESP32 GPIO 14 → 220Ω Resistor → LED 3 (Light 3) → GND
ESP32 GPIO 27 → 220Ω Resistor → LED 4 (Fan 1) → GND
ESP32 GPIO 26 → 220Ω Resistor → LED 5 (Fan 2) → GND
ESP32 GPIO 34 (ADC) ← ACS712 Analog Out (current sense)

Power: ESP32 VIN → 5V supply, GND → common ground
WiFi: ESP32 connects to WiFi → sends HTTP POST to backend API
```

ESP32 Arduino Sketch (for Wokwi simulation)

```
#include <WiFi.h>
#include <HTTPClient.h>

// Pin definitions for one room
const int LIGHT_PINS[] = {13, 12, 14};
const int FAN_PINS[] = {27, 26};
const int CURRENT_PIN = 34; // ADC for ACS712

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) pinMode(LIGHT_PINS[i], INPUT);
  for (int i = 0; i < 2; i++) pinMode(FAN_PINS[i], INPUT);

  // WiFi.begin("Wokwi-GUEST", "");
  // while (WiFi.status() != WL_CONNECTED) delay(500);
}
```

```

void loop() {
  // Read device states
  for (int i = 0; i < 3; i++) {
    bool lightOn = digitalRead(LIGHT_PINS[i]);
    Serial.printf("Light %d: %s\n", i+1, lightOn ? "ON" : "OFF");
  }
  for (int i = 0; i < 2; i++) {
    bool fanOn = digitalRead(FAN_PINS[i]);
    Serial.printf("Fan %d: %s\n", i+1, fanOn ? "ON" : "OFF");
  }

  // Read current sensor (analog)
  int raw = analogRead(CURRENT_PIN);
  float voltage = raw * (3.3 / 4095.0);
  float current = (voltage - 2.5) / 0.185; // ACS712 sensitivity
  Serial.printf("Current: %.2f A\n", current);

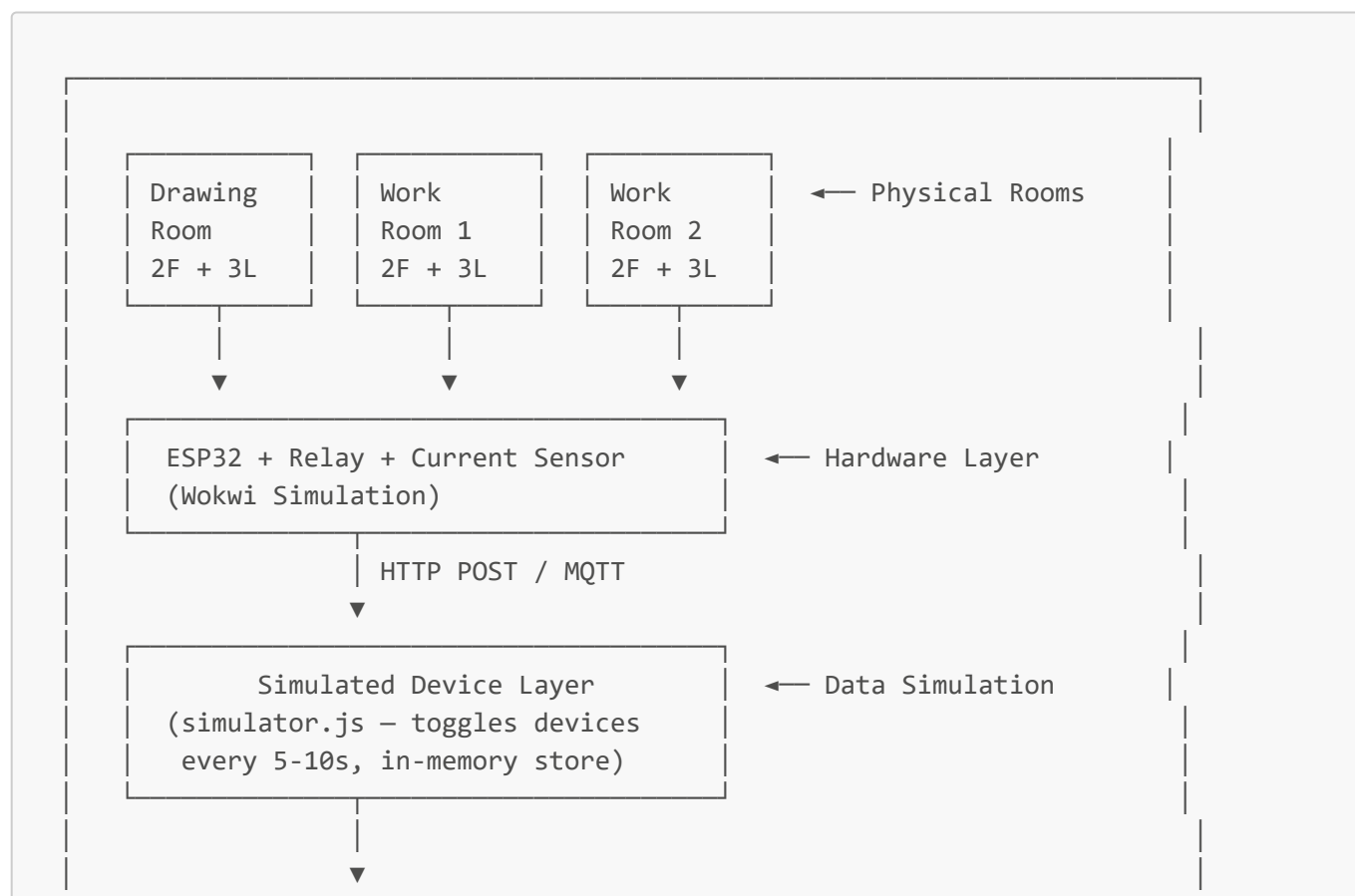
  // In real scenario: POST data to backend API via HTTP
  delay(5000);
}

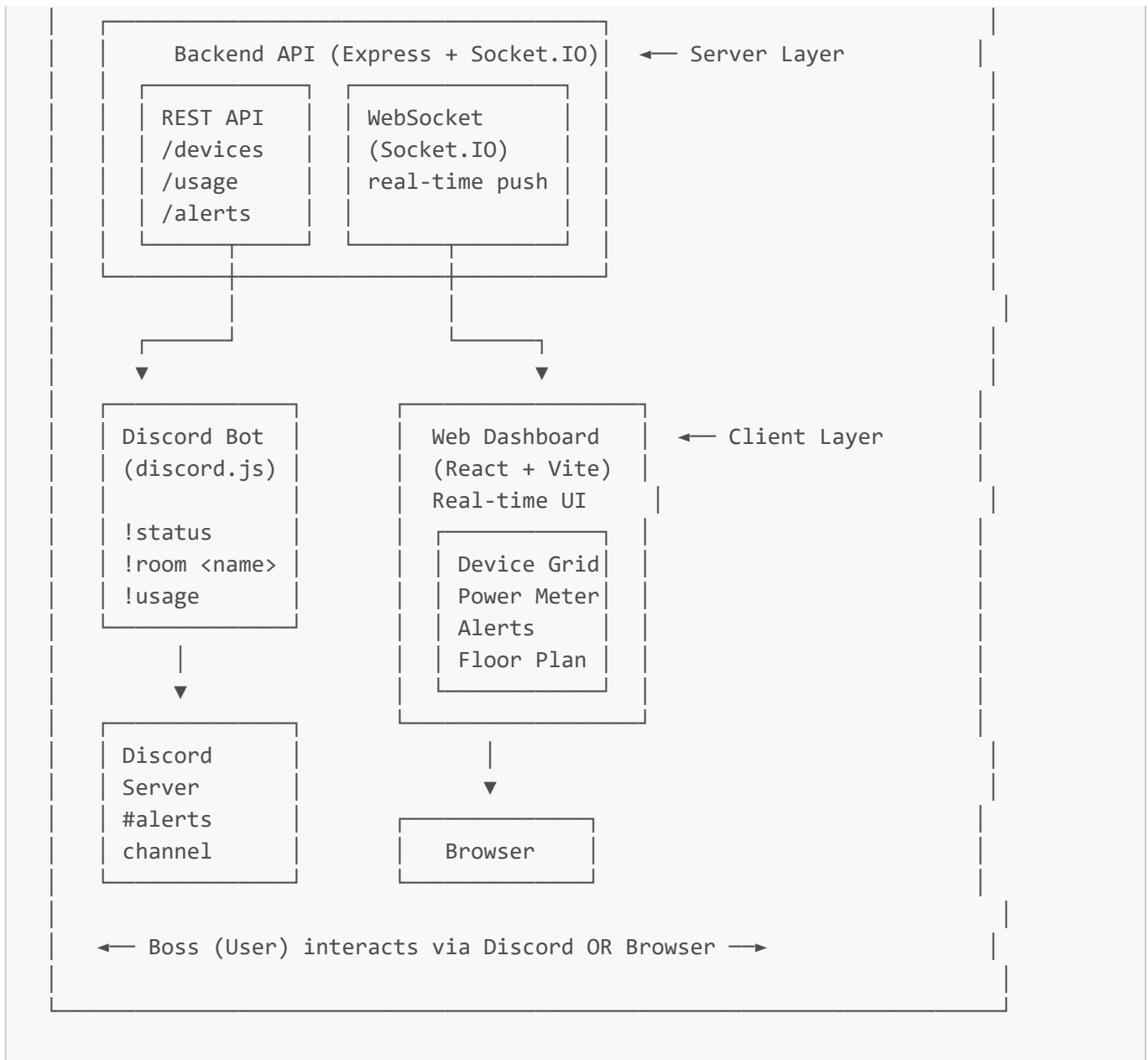
```

Step 5: System Diagram

Priority: MEDIUM — Worth 15%. Use draw.io, Excalidraw, or Figma (NOT Mermaid — explicitly banned).

What to Include





Tool Suggestion: Use **Excalidraw** (free, clean, hand-drawn style) or **draw.io** to recreate this as a professional diagram. Export as PNG and add to `diagrams/` folder.

Step 6: Dummy Data (Required Values)

As specified in the problem, use these **exact** dummy contact values:

```
[
  {
    "name": "Nafisa Rahman",
    "email": "nafisa.rahman@yahoo.com",
    "phone": "+8801812345678"
  },
  {
    "name": "Tanvir Hossain",
    "email": "tanvir.hossain@yahoo.com",
    "phone": "+8801912345678"
  }
]
```

```
    }  
  ]  
}
```

⚠ **NEVER** substitute, modify, or invent additional dummy values unless explicitly instructed.

🕒 Time Management Strategy (Hackathon Execution Order)

Order	Task	Estimated Time	Weight
1	Backend + Device Store + Simulator	1.5 hours	Foundation
2	Web Dashboard (React + Socket.IO)	2.5 hours	30%
3	Discord Bot	1 hour	10%
4	System Diagram (Excalidraw/draw.io)	30 min	15%
5	Wokwi Circuit Schematic	45 min	15%
6	Polish, README, commits	45 min	15%
7	Record 3-min demo video	30 min	Deliverable

🏆 Winning Strategies — Stand Out from the Crowd

1. Dashboard Visuals (10% weight, easy to win)

- Use a **dark theme** with glassmorphism panels
- Add **animated fan blades** (CSS rotate) and **glowing lights** (box-shadow)
- Include a **top-view floor plan** SVG with live device states — this is the BONUS feature that will make judges say "wow"

2. Discord Bot Personality

- Don't send robotic data dumps. Use **conversational, friendly tone**:
 - ☒ "Hey boss! 🙌 Drawing Room is looking good — everything's off. But Work Room 2 still has 2 fans and all 3 lights blazing!"
 - ☒ "Drawing Room: 0 fans on, 0 lights on. Work Room 2: 2 fans on, 3 lights on."
- **BONUS**: Use an LLM (OpenAI API) to generate the friendly message from raw data

3. Code Quality (15% weight)

- **Commit often** with descriptive messages (`git commit -m "feat: add real-time WebSocket updates to dashboard"`)
- Follow **conventional commits** format: `feat:`, `fix:`, `docs:`, `refactor:`
- Add JSDoc comments to key functions
- Keep a clean, detailed `README.md`

4. Demo Video Tips

- Keep it under 3 minutes
- Show the **dashboard live updating** (device states changing in real-time)
- Show the **Discord bot** responding to all 3 commands
- Briefly explain the **architecture** (point to the system diagram)
- End with the **circuit schematic** — explain the wiring logic



Verification Checklist

- ☐ Backend starts on `localhost:4000`
- ☐ `GET /api/devices` returns all 18 devices with correct structure
- ☐ `GET /api/usage` returns total watts + per-room + estimated kWh
- ☐ `GET /api/alerts` returns appropriate alerts
- ☐ Simulator toggles devices every 5-10 seconds
- ☐ Dashboard connects via WebSocket and updates live (no page refresh)
- ☐ Dashboard shows all 3 panels: Device Status, Power Meter, Alerts
- ☐ Discord `!status` shows room-by-room summary from live data
- ☐ Discord `!room work1` shows specific room details
- ☐ Discord `!usage` shows current watts and estimated kWh
- ☐ System diagram is included (NOT Mermaid)
- ☐ Circuit schematic (Wokwi/Tinkercad) for 1 room is included
- ☐ README has clear setup instructions
- ☐ Codebase has meaningful commit history
- ☐ Demo video < 3 minutes



Quick Start Commands

```
# 1. Start Backend
cd backend
npm install
node server.js

# 2. Start Frontend (new terminal)
cd frontend
npm install
npm run dev

# 3. Start Discord Bot (new terminal)
cd discord-bot
npm install
node bot.js
```

Remember: The judges are evaluating **engineering process**, not just speed. Show clean architecture, modular design, clear docs, and a polished demo. Good luck! 🍀